

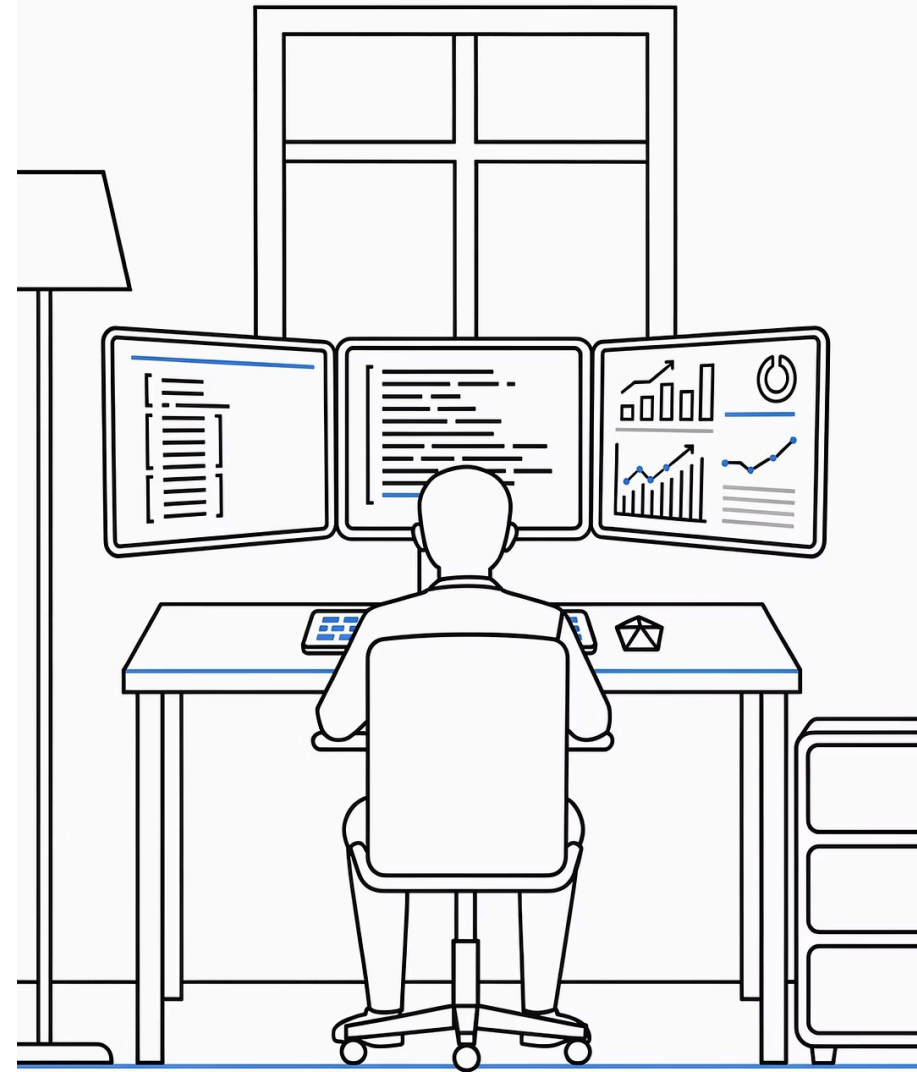
COMP 3304

SPRING 2026

CI/CD Pipelines - Continuous Integration

Fundamentals of Software Engineering

Suphi Uçar, PhD.



What Is Continuous Integration?

CI is the practice of merging all developer working copies to a shared mainline **frequently — at least daily**. Each merge automatically triggers a build and full test run.

Core principle: if it hurts, do it more often. The pain of integration decreases dramatically when you integrate constantly instead of sporadically.

CI Goals at a Glance

1 Catch integration problems early

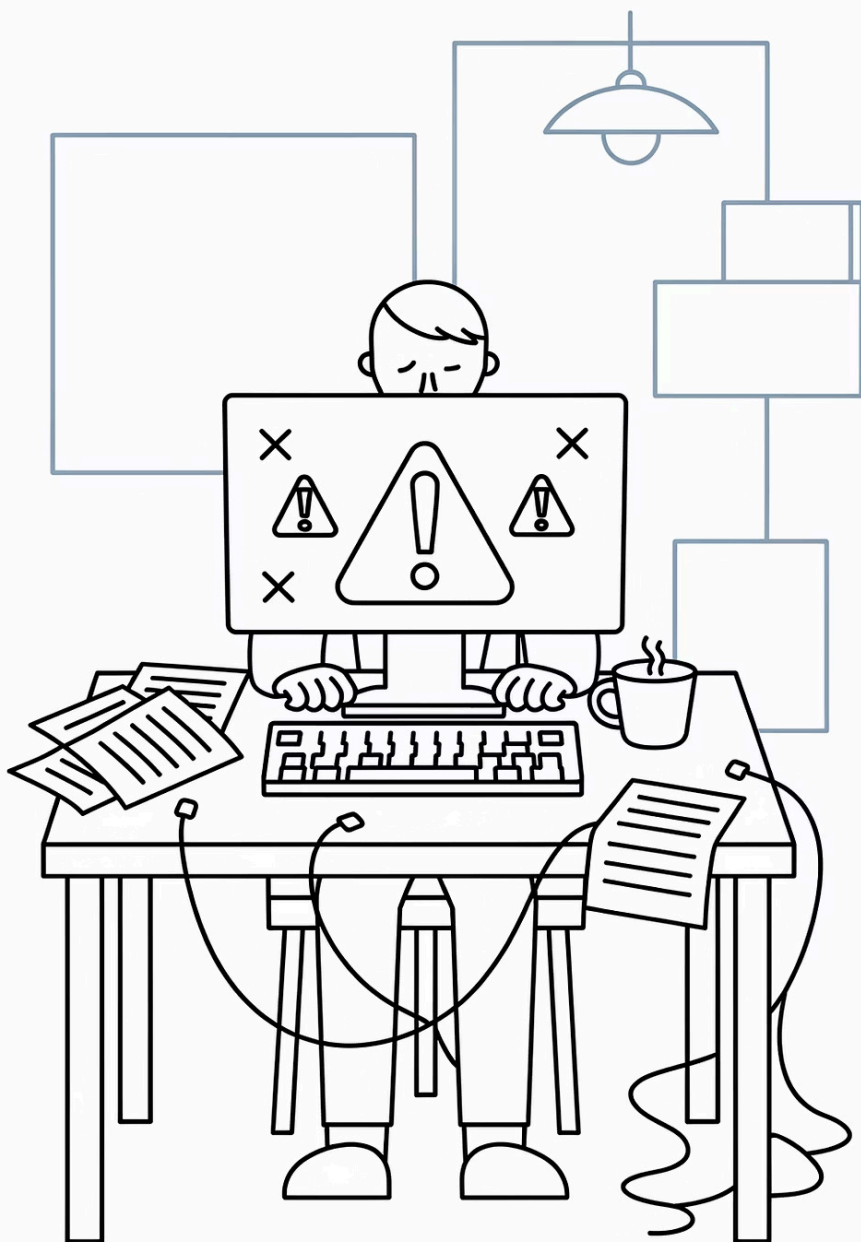
Find conflicts and breakages within minutes of a commit, not days before a release.

2 Maintain a releasable codebase

The main branch should always be in a state that could, in theory, ship to users.

3 Give developers fast feedback

Developers learn about breakages while the context is still fresh in their minds.



The Problem Before CI/CD

"It Works on My Machine"

The most dreaded phrase in software. Environment differences — OS, runtime versions, config files — make builds non-reproducible and debugging a nightmare.

Manual Build & Deploy Steps

Humans running scripts by hand are slow, inconsistent, and forget steps. One missed command can bring down production.

Integration Hell

When two teams merge a week's worth of diverged code simultaneously, resolving conflicts becomes a multi-day ordeal that blocks the whole team.

No Shared Visibility

Without automation, nobody knows at any given moment whether the codebase actually builds and passes tests — until it's too late.

A Brief History — How We Got Here

Pre-CI Era

Nightly builds, dedicated "build engineers," and release-day panic. Integration was a scheduled event, not a continuous process.

1

2010s — Hosted CI

Travis CI, CircleCI, and GitLab CI brought CI-as-a-service. No servers to manage — just connect your repo and go.

3

2000s — Early CI

CruiseControl (2001) and **Jenkins** (2011) pioneered automated builds triggered by commits. Powerful, but required heavy self-hosting and maintenance.

2

Today — Native CI

GitHub Actions, GitLab CI, and Bitbucket Pipelines embed CI directly into the platform. It's a baseline expectation, not a luxury add-on.

4

Continuous Delivery vs. Continuous Deployment

Both build on CI, but they differ in one critical way: **who or what pulls the trigger on production.**

Continuous Delivery

Every passing build is in a **deployable state**, but pushing to production is a deliberate, human-triggered decision. Teams can deploy on their own schedule — daily, weekly, or whenever stakeholders are ready.

Best for: regulated industries, enterprise software, teams with compliance requirements or staged rollout needs.



Continuous Deployment

Every passing build is **automatically pushed to production** — no human in the loop. Requires extremely high test coverage and confidence in the pipeline.

Best for: high-velocity SaaS products, teams with mature observability and rollback capabilities.

- ❏ Which is right for your team? It depends on **risk tolerance**, compliance requirements, and team maturity. Most teams start with Continuous Delivery and grow into Continuous Deployment.

The CI/CD Value Proposition

200×

More Deployments

DORA research: high-performing teams deploy **200× more frequently** than low performers — with **24× faster** recovery times.

<1hr

Feedback Loop

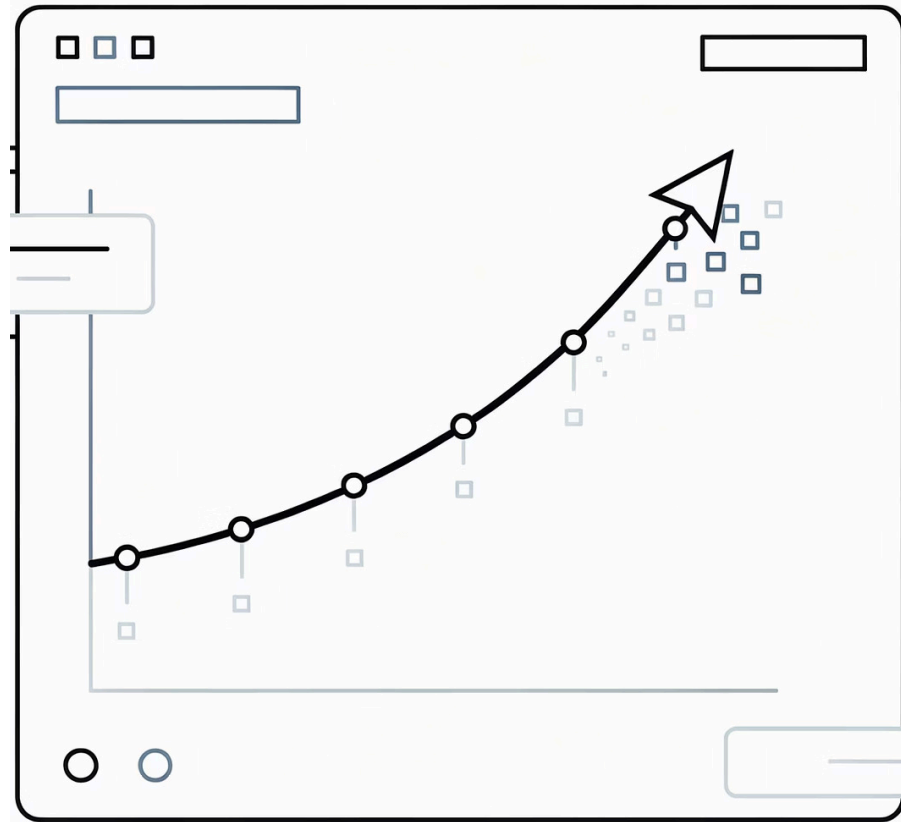
Elite teams go from commit to feedback in under an hour. Without CI, that feedback can take days.

~0

Release Day Stress

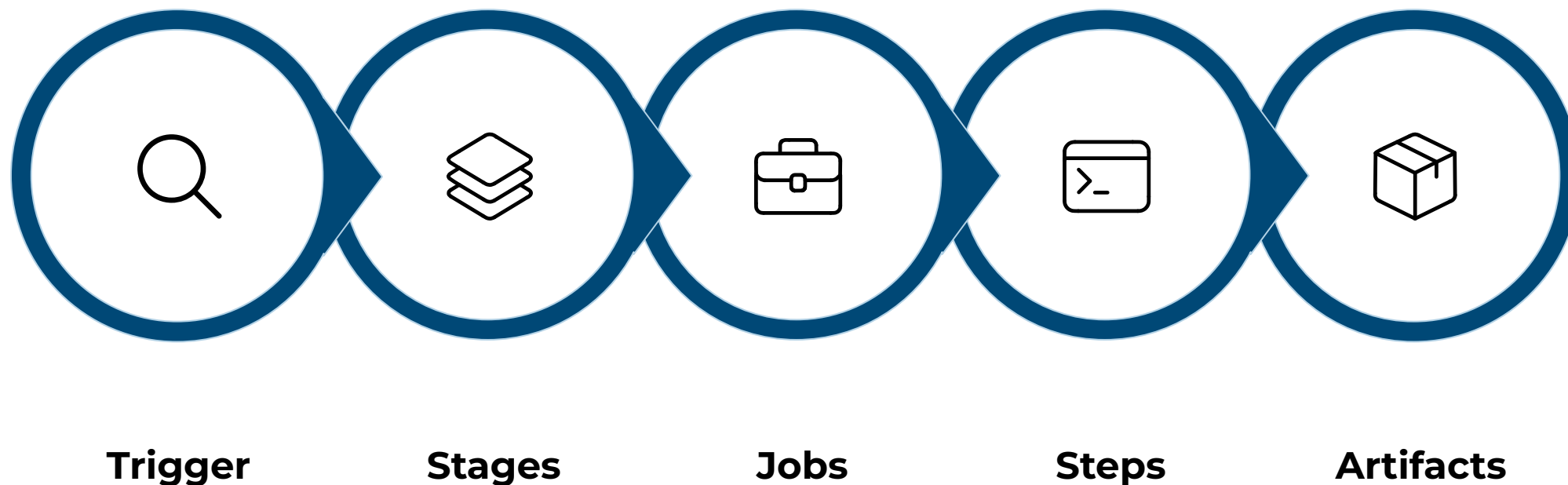
When you deploy dozens of times per day, each deployment is tiny and low-risk. "Release day" simply disappears.

The virtuous cycle: **automate → gain confidence → ship more → automate more**. Each loop makes the next one easier and faster.



Pipeline Anatomy — The Big Picture

A pipeline is a **series of automated steps triggered by an event** — think of it as a software assembly line where each station must pass quality control before the next begins.



Every element in the pipeline has a purpose. Understanding how they nest — and how data flows between them — is the key to designing pipelines that are fast, reliable, and debuggable.

Pipeline Anatomy — Triggers

A trigger is the **event that kicks off your pipeline**. Choosing the right trigger for the right job prevents wasted runner minutes and keeps feedback loops tight.



Push to a Branch

Fire on every push to `develop` or `main`. The most common trigger — ensures every commit is validated immediately.



Pull Request Events

Trigger when a PR is opened, updated, or merged. Ideal for running the full test suite before code can be approved.



Scheduled (Cron)

Use cron syntax for nightly builds, dependency audits, or long-running regression suites that would be too slow to run on every push.



Tag / Release Creation

Trigger deployment pipelines only when a version tag is pushed — a clean separation between development CI and production CD.



Manual Dispatch

`workflow_dispatch` in GitHub Actions allows a human to trigger any pipeline on demand — useful for hotfixes and one-off deployments.

Pipeline Anatomy — Stages, Jobs & Steps

1

Stage

A **logical phase** of the pipeline — e.g., `build`, `test`, `lint`, `deploy`. Stages run sequentially; a failed stage halts subsequent ones.

2

Job

A **unit of work** inside a stage. Jobs within the same stage can run **in parallel**, speeding up the overall pipeline.

3

Step

A **single command or action** inside a job — runs sequentially. This is where the actual work happens: `npm install`, `pytest`, `docker build`.

Concrete Example

Build Stage

- └─ **Job:** `install-deps`
 - └─ **Step:** `pip install -r requirements.txt`

Test Stage

- └─ **Job:** `run-tests`
 - └─ **Step:** `pytest --cov=src`

Lint Stage

- └─ **Job:** `lint-code`
 - └─ **Step:** `flake8 src/`
 - └─ **Step:** `black --check src/`

Each stage gate-keeps the next. If `pytest` fails, the lint stage never runs — saving time and surfacing the most important failure first.

Pipeline Anatomy — Artifacts

Artifacts are **files produced by a job** and either passed to later stages or stored for post-run download. They are the connective tissue between pipeline stages.

→ Why Artifacts Matter

Avoid re-running expensive steps. If you build a Docker image in the build stage, artifact it — don't rebuild it in the deploy stage.

→ Traceability

Every build produces a downloadable coverage report, binary, or log file — making debugging failed builds much easier after the fact.

Common Artifact Types

- Compiled binaries or JAR files
- HTML test coverage reports
- Docker images pushed to a registry
- Build logs for audit trails

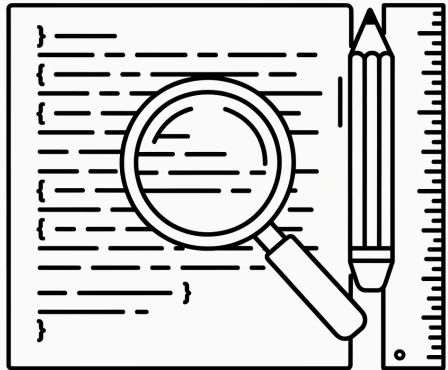
GitHub Actions Syntax

```
- uses: actions/upload-artifact@v4
  with:
    name: coverage-report
    path: htmlcov/

- uses: actions/download-artifact@v4
  with:
    name: coverage-report
```

Linting & Formatting as Pipeline Gates

Before code ever reaches a human reviewer, automated tools ensure it adheres to quality and style standards, acting as crucial gates in your pipeline.



What Are They?

Linting

Static code analysis that identifies programmatic errors, potential bugs, stylistic issues, and suspicious constructs without executing the code.

Formatting

Automated adjustment of code style (e.g., indentation, line breaks, spacing) to meet predefined rules, ensuring consistency across the entire codebase.

Why Integrate Them?



Catch Errors Early

Identify potential bugs and logical flaws immediately upon commit, reducing time spent debugging later.



Enforce Standards

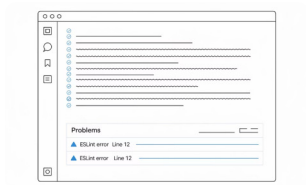
Ensure all code adheres to team-wide style guides and best practices automatically, reducing cognitive load in code reviews.



Improve Readability

Consistently formatted code is easier to read, understand, and maintain for all developers.

Linting & Formatting as Pipeline Gates



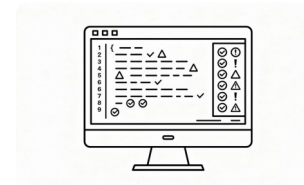
JavaScript / TypeScript

ESLint catches logic bugs and style violations. **Prettier** enforces formatting. Together: readable, consistent JS across the whole team.



Python

Flake8 and **Pylint** flag style and logic issues. **Black** auto-formats code. The pipeline runs `black --check` — format or fail.



Java

Checkstyle enforces naming conventions, documentation requirements, and formatting rules defined by your team or Google's style guide.

Adding a lint stage after install — but before or alongside tests — means **style debates leave code review and move into the tool**. Consistent style is enforced automatically, not by convention.

Introduction to GitHub Actions

Native to GitHub

No external CI service needed. Actions is built directly into GitHub — your repo, your workflows, one platform.

YAML-Defined Workflows

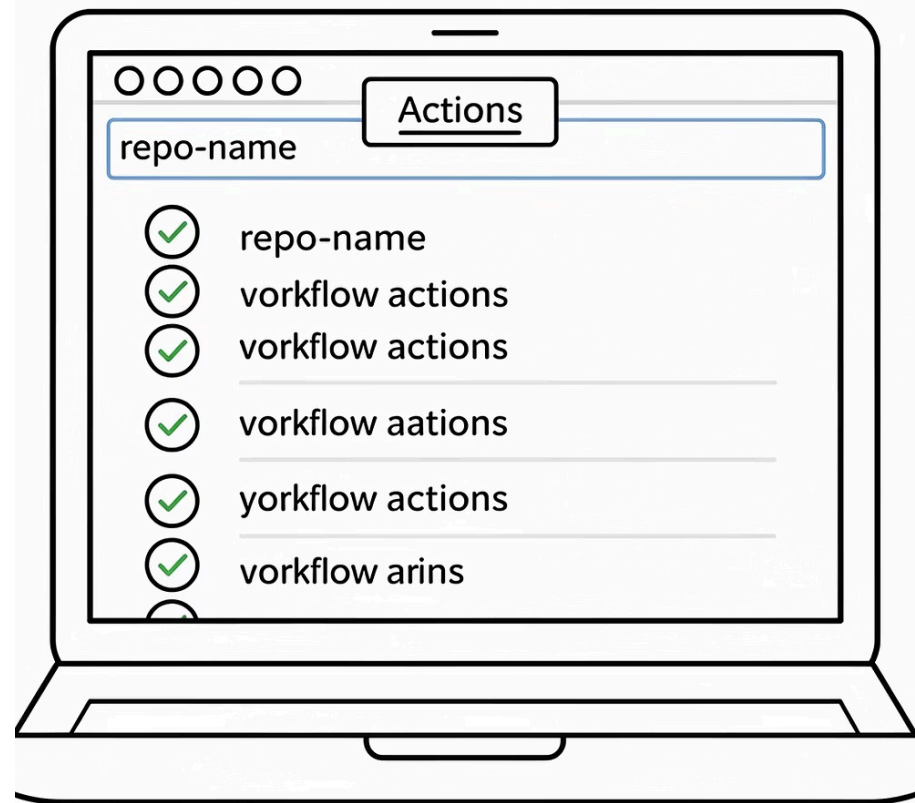
Workflows live in `.github/workflows/` as `.yaml` files — version-controlled alongside your code, reviewable in PRs.

GitHub-Hosted Runners

Run on Ubuntu, Windows, or macOS without managing infrastructure. Self-hosted runners available for custom needs.

Generous Free Tier

Public repos get unlimited minutes. Private repos receive a monthly quota — more than enough for course projects.



YAML Syntax

GitHub Actions workflows are defined using YAML (YAML Ain't Markup Language), a human-readable data serialization standard. Understanding its core syntax is crucial for building robust pipelines.

Key-Value Pairs

The fundamental building blocks, associating a unique key with a value. Example: `name: "My Workflow"`.

Lists (Sequences)

Represented by hyphens, creating an ordered collection of items. Example:

```
- step 1  
- step 2.
```

Mapping (Objects)

Nested key-value pairs defined by indentation, forming hierarchical structures. Example:

```
jobs:  
  build:.
```

Indentation Matters

YAML uses consistent indentation (spaces, never tabs) to define structure and relationships between elements. Precision is key.

This simple structure allows complex workflows to be defined clearly and concisely:

```
name: My First Workflow  
  
on: [push]  
  
jobs:  
  build:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - name: Run a one-line script  
        run: echo Hello, world!  
      - name: Run a multi-line script  
        run: |  
          echo Add other actions to build,  
          echo test, and deploy your project.
```

GitHub Actions — YAML Syntax Deep Dive

Annotated ci.yml

```
name: CI Pipeline      # Workflow name

on:                    # Triggers
  push:
    branches: [develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest # Runner OS

    steps:
      - uses: actions/checkout@v4 # Fetch code
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install -r requirements.txt
      - run: pytest --cov=src
    env:
      API_KEY: ${ secrets.API_KEY } # Safe secret ref
```

Key Concepts

on: — Triggers

Defines which GitHub events activate the workflow. Can be a single event or a list.

Marketplace Actions

actions/checkout,
actions/setup-node,
actions/setup-python —
community-maintained,
versioned building blocks.

Secrets

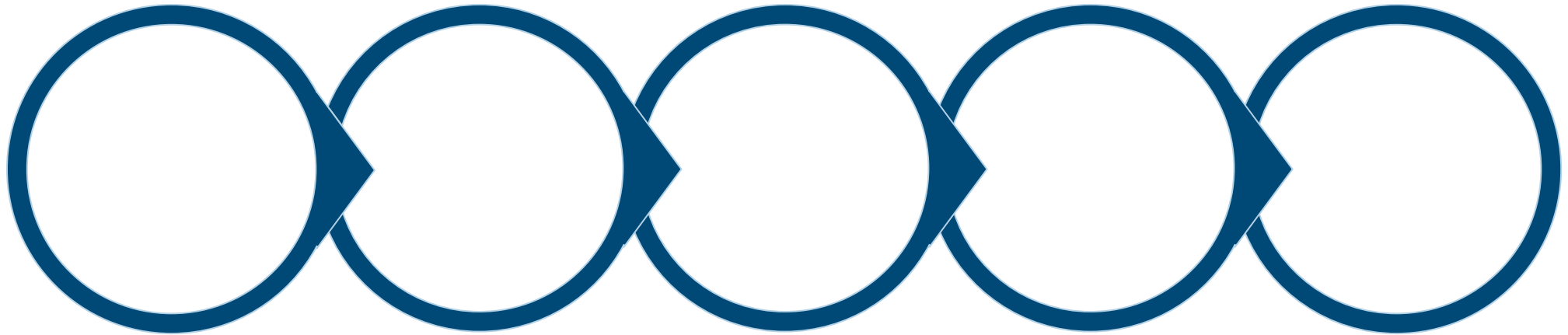
Reference via `${ secrets.MY_SECRET }`. **Never**
hardcode credentials in
YAML — they end up in your
git history forever.

env: Scoping

Set environment variables at
the workflow, job, or step
level. Narrower scope =
better security.

Live Walkthrough — A Real CI Pipeline

Let's trace a complete pipeline for a Python project from the moment a developer pushes code to the green checkmark.



Triggers Used

Push to `develop` and any Pull Request targeting `main` — ensuring all integration points are validated.

Common Gotchas

- Forgetting `actions/checkout` — your code isn't on the runner yet!
- Wrong working directory when using monorepos
- Missing environment variables that exist locally but not in CI

Automated Testing in the Pipeline

The test suite from Week 10 wasn't just an assignment — it was preparation for this moment. Now, every single push triggers it automatically.



Failing Tests = Failed Pipeline

A broken test blocks the PR from merging. No overrides, no "I'll fix it later." The code simply cannot proceed.



Fail Fast

The pipeline stops at the first failing step. You get immediate, specific feedback — not a cryptic failure at the end of a 20-minute run.



Coverage Enforcement

The 70% coverage floor from Week 10 becomes a hard pipeline gate. Drop below it and the build fails — automatically.

📄 **The result:** broken or untested code is **structurally unable** to reach `main`. Not by convention — by automation.

Blocking Merges — Branch Protection Rules

How to Configure

1. Go to **Repository Settings**
2. Navigate to **Branches**
3. Click **Add branch protection rule**
4. Set the pattern to `main`
5. Enable "**Require status checks to pass before merging**"
6. Select your CI job name from the list
7. Optionally require at least one approving review

Why This Matters

Branch protection rules transform `main` from a branch developers can accidentally break into a **protected, always-green deployment candidate**.

Combining required CI checks with the PR review workflow from Week 9 creates a complete quality gate: **automated correctness + human judgment**, both enforced before any code merges.

📌 End state: `main` is always green, always deployable — no exceptions.

Pipeline Best Practices

1

Stay Under 10 Minutes

Feedback loops longer than 10 minutes break developer flow. Optimize ruthlessly — parallelize, skip unnecessary steps.

2

Cache Dependencies

Use `actions/cache` to store `node_modules` or pip packages. Avoid reinstalling on every run — it's the single biggest speedup available.

3

Matrix Builds

Test against multiple runtime versions in parallel. A library that claims to support Python 3.10–3.13 should prove it in CI.

4

Never Commit Secrets

Use GitHub Secrets, environment variables, or a dedicated secrets manager. Credentials in git history are permanently compromised.

5

Keep YAML DRY

Reusable workflows and composite actions eliminate copy-paste. If you're repeating a block of YAML, abstract it.

6

Add a Status Badge

A pipeline badge in `README.md` signals health at a glance — to teammates, contributors, and future employers reviewing your portfolio.

Troubleshooting CI Failures

Common Failure Categories

Dependency Issues

Package not installed, wrong version pinned, or missing from `requirements.txt` / `package.json`.

Missing Environment Variables

Config that exists in your `.env` file locally doesn't exist on the runner. Add it to GitHub Secrets.

Flaky Tests

Tests that pass locally but fail in clean CI environments — often due to ordering assumptions, timing, or external service calls.

Runner OS Assumptions

Path separators, shell differences (`bash` vs. `cmd`), and case sensitivity vary by OS. Test on the right runner.

Debugging Techniques

01

Read the Step Logs

GitHub Actions shows step-by-step output. Expand each step — the error is almost always in the last few lines of a failing step.

02

Add `echo` & `env` Dumps

Temporarily add `run: env` or `run: echo $VAR` steps to inspect the runner's state at any point in the pipeline.

03

Use `act` Locally

The `act` tool runs GitHub Actions workflows locally using Docker — fast iteration without pushing commits to get feedback.

04

SSH via `tmate`

Add a `tmate` action step to get live SSH access into the runner mid-pipeline — the nuclear option for hard-to-reproduce failures.

Working CI pipeline

- Work as a team to create a live, working CI pipeline using Github Actions.
- Use the "buggy" [cart.py](#) file from the last weeks assignment.
- **Requirements:**
 - Triggers on every push to `develop`
 - Runs the full test suite automatically
 - Blocks merging to `main` if any tests fail (branch protection rule must be enabled)
 - Add a linting or formatting step as an additional gate
- **Deliverable:** Report file (pdf) which includes these parts:
 - Screen shots
 - YML file

 FURTHER READING

References

To deepen your understanding of software architecture and explore specific styles in more detail, consult the following resources:

- Engineering Software Products - An Introduction to Modern Software Engineering, Global Ed., Ian Sommerville, Pearson, 2021.